

Question - 1

Q1] prove Fermat's Little Theorem from group-theoretic first principle, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base=2) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

6 minutes

⇒ Fermat's little theorem:

If p is a prime number and a is not divisible by p then $a^{p-1} \equiv 1 \pmod{p}$

proof:

The numbers $1, 2, 3, \dots, p-1$ are all different modulo p .

Multiplying each by a we get,

$a, 2a, 3a, \dots, (p-1)a$

These are also the same set of numbers in a different (order).

$$\text{So, } a^{p-1} (p-1)! \equiv (p-1)! \pmod{p}$$

cancel $(p-1)!$:

$$a^{p-1} \equiv 1 \pmod{p}$$

Miller-Rabin Test:

Fermat's theorem says that if n is prime,

$$a^{n-1} \equiv 1 \pmod{n}$$

This idea is used in Miller-Rabin Test to check whether a number is probably prime. Miller-Rabin is stronger than simple Fermat test because it checks the powers of a more carefully.

$$\text{for } n=561, a=2$$

$$561-1=560$$

Fermat test gives,

$$2^{560} \equiv 1 \pmod{561}$$

So, the simple Fermat test may say 561 is prime.

$$\text{But } 561 = 3 \times 11 \times 17$$

Therefore, 561 is composite

561 is called a Carmichael number because it is composite but still satisfies $a^{560} \equiv 1 \pmod{561}$ for every a relatively prime to 561.

Therefore 561 can fool the simple Fermat test, while Miller-Rabin is designed to detect such composite numbers.

Question-2

Q2] prove that Z_n under addition and multiplication modulo n , is a field if and only if n is prime. Illustrate your proof by constructing the multiplicative structure of Z_{11} (prime module) with Z_{12} (composite module).

7 minutes

⇒ proof

Let $Z_n = \{0, 1, 2, \dots, n-1\}$
with addition and multiplication modulo n .

If n is prime

If n is prime, every non-zero element of Z_n has a multiplicative inverse.

For example, in Z_{11} :

$$2 \times 6 = 12 \equiv 1 \pmod{11}$$

So the inverse of 2 is 6.

Similarly every non zero element has an inverse.

Therefore Z_n is a field.

If n is composite:

$$n = ab$$

where $1 < a, b < n$.

$$\text{Thus } a \times b = 0 \pmod{n}$$

Section-2

even though $a \neq 0$ and $b \neq 0$. So, $\mathbb{Z}_n$ has zero divisors and non-zero elements may not have inverse.

For $\mathbb{Z}_{12}$:

(Example F)

$$3 \times 4 = 12 \equiv 0 \pmod{12}$$

here $3 \neq 0$, $4 \neq 0$, but their product is 0.

Therefore $\mathbb{Z}_{12}$ is not a field.

Comparison:

- $\mathbb{Z}_{11} : 11$ is prime $\rightarrow$ every non-zero element has an inverse $\rightarrow$ field.
- $\mathbb{Z}_{12} : 12$ is composite $\rightarrow$ has zero divisors $\rightarrow$ not a field.

$\therefore \mathbb{Z}_n$ is a field if and only if n is prime.

If n is composite

$$n = ab$$

$$1 < a < n, 1 < b < n$$

$$(n \text{ mod } a) \cdot b = 0 \pmod{n}$$

Question-3

Q3] construct $GF(2^3)$ using the irreducible polynomial x^3+x+1 over $GF(2)$. Build the multiplication table for a generator element α , and determine whether the multiplicative group $GF(2^3)$ is cyclic.

5 minutes

$\Rightarrow$ Let, $f(x) = x^3+x+1$

since $f(x)$ is irreducible over $GF(2)$, we can construct.

$GF(2^3) = GF(2)[x] / (x^3+x+1)$

Let, α be the root of $f(x)$. Then,

$\alpha^3 + \alpha + 1 = 0$

$\Rightarrow \alpha^3 = \alpha + 1$

The elements are,

$GF(8) = \{0, 1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$

Powers of α (using $\alpha^3 = \alpha + 1$)

$\alpha^0 = 1$

$\alpha^1 = \alpha$

$\alpha^2 = \alpha^2$

$\alpha^3 = \alpha + 1$

$\alpha^4 = \alpha(\alpha + 1) = \alpha^2 + \alpha$

$\alpha^5 = \alpha^2 + \alpha + 1$

$\alpha^6 = \alpha^2 + 1$

$\alpha^7 = 1$

Ques-10-3

Therefore the non-zero elements are:

$$1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6$$

The multiplication rule is: $a^i \cdot a^j = a^{(i+j) \bmod 7}$

So, the multiplication table:

X	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

Since $\alpha^7 = 1$

and no smaller positive power of α is 1, the order of α is 7.

The multiplicative group has,

$$|GF(8)^*| = 8-1 = 7 \text{ elements.}$$

Since α has order 7, it generates the whole group.

Therefore $GF(2^3)^* = GF(8)^*$ is cyclic.

Question 4

Q41 Derive the time complexity of modular exponentiation via square and multiply. Compute $7^{560} \bmod 561$ step by step, and interpret the result in light of 561 being a carmichael number.

5 minutes

⇒ Square-and-Multiply:

It's a fast method for calculating $a^e \bmod n$.

Instead of multiplying a e times, we repeatedly square the result. Therefore the number of operations is proportional to the number of bits in e .

If e has k bits, the time complexity is $O(\log e)$.

Calculate $7^{560} \bmod 561$.

We have, $560 = 512 + 32 + 16$

Now calculate by repeated squaring:

$$7^1 = 7 \pmod{561}$$

$$7^2 = 49$$

$$7^4 = 49^2 = 167 \pmod{561}$$

$$7^8 = (167)^2 \equiv 526 \pmod{561}$$

$$7^{16} = (526)^2 \equiv 103 \pmod{561}$$

$$7^{32} = (103)^2 \equiv 511 \pmod{561}$$

$$7^{64} = (511)^2 \equiv 256 \pmod{561}$$

Question 1

$$7^{128} \equiv (266)^2 \equiv 460 \pmod{561}$$

$$7^{256} \equiv 103 \pmod{561}$$

$$7^{512} \equiv 511 \pmod{561}$$

$$\therefore 7^{560} \equiv 511 \times 511 \times 103 \pmod{561} \\ \equiv 1 \pmod{561}$$

Interpretation of the result:

$$561 = 3 \times 11 \times 17$$

So, 561 is a composite number but it's a Carmichael number.

$$\text{Since, } 7^{560} \equiv 1 \pmod{561}$$

Therefore 561 can pass the simple Fermat primality test even though it's composite.

$$7^1 \equiv 7 \pmod{561}$$

$$7^2 \equiv 49 \pmod{561}$$

$$7^4 \equiv 2401 \equiv 127 \pmod{561}$$

$$7^8 \equiv (127)^2 \equiv 16129 \equiv 520 \pmod{561}$$

$$7^{16} \equiv (520)^2 \equiv 270400 \equiv 103 \pmod{561}$$

$$7^{32} \equiv (103)^2 \equiv 10609 \equiv 511 \pmod{561}$$

$$7^{64} \equiv (511)^2 \equiv 261121 \equiv 1 \pmod{561}$$

Question 6

Q6] compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048 bit and 4096-bit key sizes.

⇒ comparison between deterministic vs probabilistic primality Testing.

Feature	AKS	Miller-Rabin
Type	Deterministic	probabilistic
Result	Always correct	probably correct
complexity	polynomial time, about $O((\log n)^6)$ in the original AKS	About $O(k(\log n)^3)$ for k rounds
Error	No error	Error can be made extremely small.
practical use	Mainly theoretical	widely used in practice

AKS:

AKS can determine whether a number is prime with 100% certainty. However, it is relatively slow for very large numbers.

Miller-Rabin

Miller-Rabin is much faster. For each round, a composite number has a small chance of passing the test. For one suitable round, the error probability is at most $\frac{1}{4}$.

After k rounds: $(\frac{1}{4})^k$

so, more rounds make the error probability extremely small.

For RSA, large prime numbers are needed. At 2048-bit and 4096-bit key sizes, many candidate numbers must be tested. Therefore Miller-Rabin is more practical as it is fast and its error probability can be made negligible.

practical use	No error	Error can be made extremely small
mainly theoretical		widely used in practice

AKS can determine whether a number is prime with certainty. However, it is relatively slow for very large numbers.

Question 6

Q6] Derive the extended Euclidean Algorithm formally (as a sequence of linear combinations), then use it to compute $\text{gcd}(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \text{gcd}(240, 46)$

6 minutes

⇒ Euclidean algorithm:

$$a = q_1 b + r_1, \quad b = q_2 r_1 + r_2, \quad r_1 = q_3 r_2 + r_3, \dots$$

until remainder = 0, The last nonzero remainder is gcd .

The extended euclidean algorithm finds

$$\text{gcd}(a, b) = ax + by$$

first use the Euclidean algorithm:

$$a = q_1 b + r_1$$

$$b = q_2 r_1 + r_2$$

continue until the remainder becomes 0. The last non-zero remainder is gcd .

Compute $\text{gcd}(240, 46)$

Divide repeatedly:

$$\begin{aligned} 240 &= 5(46) + 10 \\ 46 &= 4(10) + 6 \\ 10 &= 1(6) + 4 \\ 6 &= 1(4) + 2 \\ 4 &= 2(2) + 0 \end{aligned}$$

The last non zero remainder = 2:

$$\therefore \gcd(240, 46) = 2$$

Back substitute

$$2 = 6 - 4$$

$$= 6 - (10 - 6)$$

$$= 2(6) - 10$$

$$= 2(46 - 4(10)) - 10$$

$$= 2(46) - 9(10)$$

$$= 2(46) - 9(240 - 5(46))$$

$$= 47(46) - 9(240)$$

Therefore:

$$2 = 240(-9) + 46(47)$$

So, the Bezout coefficients are, $x = -9$ and $y = 47$.

~~check~~

$$\therefore \gcd(240, 46) = 2, x = -9, y = 47.$$

$$\text{Check: } 240(-9) + 46(47) = -2160 + 2162 = 2$$

Que-7

Q7] State and prove the Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 4 \pmod{7}, x \equiv 6 \pmod{11}.$$

7 min

⇒ CRT

If $m_1, m_2, \dots, m_n$ are pairwise coprime, then the system

$$x \equiv a_1 \pmod{m_1}, \dots, x \equiv a_n \pmod{m_n}$$

has a unique solution modulo

$$M = m_1 m_2 \dots m_n.$$

proof

$$\text{Let } M = m_1 m_2 \dots m_n$$

$$\text{and } M_i = \frac{M}{m_i}$$

Since the moduli are pairwise coprime,

$$\gcd(M_i, m_i) = 1$$

So, M_i has an inverse y_i modulo m_i :

$$M_i y_i \equiv 1 \pmod{m_i}$$

$$\text{Then, } x = \sum_{i=1}^n a_i M_i y_i$$

satisfies all the congruences. Therefore, a solution exists.

uniqueness

If x and x' are two solutions, then

$$x - x' \equiv 0 \pmod{m_i}$$

for every i , since the modulo are pairwise coprime,

$$M = m_1 m_2 \dots m_n \mid (x - x')$$

Thus, $x \equiv x' \pmod{M}$

So, the solution is unique modulo M .

Solve

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 4 \pmod{7}$$

$$x \equiv 5 \pmod{11}$$

The product, $M = 3 \times 5 \times 7 \times 11 = 1155$

using CRT,

$$M_1 = \frac{1155}{3} = 385, M_2 = 231, m_3 = 165, M_4 = 105.$$

Their inverses are,

$$385 \equiv 1 \pmod{3}$$

$$231 \equiv 1 \pmod{5}$$

$$165 \equiv 4 \pmod{7}$$

$$4^{-1} = 2 \pmod{7}$$

$$M_4 = 105 \pmod{11}$$

$$\Rightarrow 105 = 11 \times 9 + 6$$

$$\Rightarrow 106 \equiv 6 \pmod{11}$$

$$\text{Need } 6y \equiv 1 \pmod{11}$$

$$6 \times 2 = 12 \equiv 1$$

$$y_4 = 2$$

$$106 \equiv 6 \pmod{11}, \quad 6^{-1} = 2 \pmod{11}$$

Therefore,

$$x = a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3 + a_4 M_4 y_4$$

$$= (2 \times 386 \times 1) + (3 \times 231 \times 1) + 4 \times 166 \times 2 + 6 \times 106 \times 2$$

$$= 3833$$

Reduce modulo 1165.

$$3833 = 3 \times 1165 + 368$$

$$x \equiv 368 \pmod{1165}$$

$$\therefore x = 368 + 1165K, \quad K \in \mathbb{Z}$$

$$9 \text{ base } 10 = \text{value}$$

$$(9 \text{ base } 10) = \text{value}$$

$$(10 \text{ base } 10) = \text{value}$$

Q81 Derive the CRT-based (RSA decryption/signing optimization from first principle) (computing d_p , d_q and q_{inv}), and demonstrate the $\sim 4 \times$ speed-up with a fully worked numerical example using small primes p, q .

7 min

$\Rightarrow$ In RSA, $c = m^e \pmod{n}$

and decryption and signing is,

$$m = c^d \pmod{n}$$

Where, $n = pq$.

Instead of calculating modulus, CRT calculate separately modulo p and q .

CRT parameters:

$$d_p = d \pmod{p-1}$$

$$d_q = d \pmod{q-1}$$

$$q_{inv} = q^{-1} \pmod{p}$$

calculate two small powers:

$$m_1 = c^{d_p} \pmod{p}$$

$$m_2 = c^{d_q} \pmod{q}$$

Then combine them using CRT:

$$h = q_{\text{inv}} (m_1 - m_2) \pmod{P}$$

$$m = m_2 + hq$$

Example

$$P=5, q=11$$

$$n=55. \text{ choose } e=3$$

$$\phi(n) = (5-1)(11-1) = 40.$$

We need $3d \equiv 1 \pmod{40}$

$$d=27.$$

$$\text{Let } c=12$$

CRT parameters

$$dp = 27 \pmod{4} = 3$$

$$dq = 27 \pmod{10} = 7$$

$$q_{\text{inv}} \equiv 11^{-1} \pmod{5}$$

$$\Rightarrow 11 \equiv 1 \pmod{5}$$

$$\therefore q_{\text{inv}} = 1$$

Calculate m_1 and m_2

$$m_1 = 12^3 \pmod{5}$$

$$= 2^3 \pmod{5}$$

$$= 8 \pmod{5}$$

$$= 3$$

$$m_2 = 12^7 \bmod 11$$

$$= 17 \bmod 11$$

$$= 1$$

$$h = q \text{inv}(m_1 - m_2) \bmod p$$

$$= 1(3-1) \bmod 5$$

$$= 2 \bmod 5 = 2$$

$$m = m_2 + hq$$

$$= 1 + 2 \cdot 11$$

$$= 23$$

check using normal RSA: $12^{27} \bmod 55 = 23$.

Why CRT is ~4x faster

Let n have $2K$ bits, so p, q each have about K bits, and dp and dq have about K bits.

Cost of modular exponentiation $\propto (\text{exponent bits}) \times (\text{modulus bits})^2$

Direct: $C^d \bmod n \longrightarrow 2K \times (2K)^2 = 8K^3$

CRT: two OPS, each K bit exponent $\times K$ bit modulus $\longrightarrow 2 \times K \times (K)^2 = 2K^3$

$$\text{Speed up} = \frac{8K^3}{2K^3} = 4$$

So, CRT-RSA is approximately 4x faster.

8-09

Q9] In RSA-CRT,

7 min

the signature is calculated separately:

$$s_p = m^{d_p} \pmod{p}$$

$$s_q = m^{d_q} \pmod{q}$$

Then they are combined using CRT.

Attack

Suppose an attacker causes a fault in one calculation, for example s_p while s_q is correct.

The resulting faulty signature s' will be correct modulo q , but incorrect modulo p .

The attacker knows the message m , the correct signature s , and the faulty signature s' .

Then $\gcd(s-s', n)$

will usually reveal one of the secret factors p and q .

For example if the faulty result is correct modulo q ,

$$s \equiv s' \pmod{q}$$

$$\text{So, } q \mid (s-s')$$

Therefore $\gcd(s-s', n) = q$.

and the attacker can factor $n = pq$.

once p and q are known, the RSA private key can be recovered.

Countermeasures:

- The main protection is to verify the signature before releasing it:

$$s^2 \bmod n \stackrel{?}{=} m$$

If the signature is faulty, verification fails and the signature is not released.

- Recomputing the signature
- using fault detection/ checks during CRT computation.
- using binding techniques.

$$(n-2) \bmod n$$

$$(p \bmod p) = 0$$

$$(1-2) \bmod 2$$

$$s = (n-2) \bmod n$$

and the attacker can recover m by $m = s^2 \bmod n$ since p and q are known the RSA private key can be recovered.

Q10

Q10] compute a modular inverse using both Extended Euclidean Algorithm and Fermat's-Little-Theorem based exponential and discuss the trade-offs between the two approaches for large moduli.

8 min

⇒ The modular inverse of a modulo m is a number x such that
 $ax \equiv 1 \pmod{m}$

Example: find $3^{-1} \pmod{7}$

Method 1 (Extended Euclidean)

$$7 = 2(3) + 1, \quad 3 = 3(1) + 0$$

$$\text{So, } 1 = 7 - 2(3)$$

$$\gcd(7, 3) = 1 \quad (\text{inverse exists})$$

Back substitute

$$1 = 7 - 2(3)$$

$$\text{therefore } 1 = (-2)3 + (1)7$$

This is in the form of $7x + 3y = 1$, $x = 1$, $y = -2$

$$3 \times (-2) \equiv 1 \pmod{7}$$

Convert -2 to positive residue $-2 + 7 = 5$.

$$3^{-1} = 5 \pmod{7}$$

$$\text{Check: } 3 \times 5 = 15 = 2(7) + 1 = 1 \pmod{7}$$

Method 2: Fermat's Little Theorem

For prime p ,

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore $a^{-1} \equiv a^{p-2} \pmod{p}$

for $a=3, p=7$:

$$3^{-1} \equiv 3^{7-2} \equiv 3^5 \pmod{7}$$

$$\text{here, } 3^5 = 243$$

$$243 \equiv 5 \pmod{7}$$

$$\text{So, } 3^{-1} = 5 \pmod{7}.$$

Comparison

— Extended Euclidean algorithm: usually faster and works for any modulus when $\gcd(a, m) = 1$.

— Speed faster — $O((\log n)^2)$.

Fermat's method: works when the modulus is prime and requires modular exponentiation.

— slower — $O((\log n)^3)$.

For large numbers, EEA is generally more efficient for finding a single modular inverse.

Q11

Q11] Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing (16 letters) with a keyword. present the complete 5x5 Key matrix as a table and show diagram substitution step.

7 min

⇒ playfair cipher:

Keyword : Monarchy

plaintext : INSTRUMENTSFORCRYPTO

5x5 key matrix

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	S	T	
U	V	W	X	Z

plaintext

IN ST RU ME NT SF OR ER YP TO

Encryption

Rules:

1. Same row $\rightarrow$ shift right.
2. Same column $\rightarrow$ shift down.
3. Rectangle swap $\rightarrow$ same row, other letters columns.

pair	encryption	result
IN	I $\rightarrow$ G, N $\rightarrow$ A	GA
ST	S $\rightarrow$ T, T $\rightarrow$ L	TL
RU	R $\rightarrow$ M, U $\rightarrow$ Z	MZ
ME	M $\rightarrow$ C, E $\rightarrow$ L	CL
NT	N $\rightarrow$ R, T $\rightarrow$ G	RG
SF	S $\rightarrow$ P, F $\rightarrow$ J	PJ
OR	O $\rightarrow$ N, R $\rightarrow$ M	NF
CR	C $\rightarrow$ D, R $\rightarrow$ M	DM
YP	Y $\rightarrow$ H, P $\rightarrow$ G	HG
TO	T $\rightarrow$ P, O $\rightarrow$ R	PR

ciphertext: GATLMZCLRGPI NFDMHGPR.

Decryption

- Same row: Shift Left
- Same column: Shift up.
- Rectangle: Self-inverse.

Cipher pair	Decryption	Plain pair
GA	$G \rightarrow J, A \rightarrow N$	IN
TL	$T \rightarrow S, L \rightarrow T$	ST
MZ	$M \rightarrow V, Z \rightarrow R$	UR
CL	$C \rightarrow M, L \rightarrow E$	ME
RQ	$R \rightarrow N, Q \rightarrow T$	NT
PJ	$P \rightarrow S, J \rightarrow F$	SF
NM	$N \rightarrow O, M \rightarrow R$	OR
DM	$D \rightarrow E, M \rightarrow R$	ER
HQ	$H \rightarrow Y, Q \rightarrow P$	YP
PR	$P \rightarrow T, R \rightarrow O$	TO

Recovered plaintext:

INSTRUMENTS FOR CRYPTO.

8.12 Weakness of playfair cipher:

playfair is stronger than monoalphabetic substitution because it encrypts two letters at a time. Therefore a single plaintext letter doesn't always produce the same ciphertext result.

However, it still has some weakness:

1. Digraph Frequency Analysis:

playfair encrypts letters in pairs (digraphs). Therefore an attacker can study the frequency of common letter pairs.

common pairs such as TH, HE, IN, ER may appear frequently in English. By analyzing repeated ciphertext digraphs and their patterns, the attacker can guess part of the plaintext and eventually discover key matrix.

2. Known - plaintext Attack:

If an attacker knows some plaintext and its corresponding ciphertext, they can compare the plaintext digraphs with ciphertext digraphs.

By this the attacker may reconstruct the key matrix.

It's stronger than monoalphabetic substitution because:

In monoalphabetic substitution, each plaintext letter always maps to the same ciphertext letter, making frequency analysis very easy.

Playfair encrypts pairs of letters, so the simple single letter frequency pattern is hidden.

So, playfair cipher is stronger than monoalphabetic substitution, but it's not secure against modern cryptanalysis.

Common pairs such as TH, IN, ER, and ED are frequently in English. By analyzing repeated ciphertext digraphs and their patterns, the attacker can guess part of the plaintext and eventually discover the key matrix.

3. Known-plaintext Attack:

If an attacker knows some plaintext and its corresponding ciphertext, they can compare the plain digraphs with ciphertext digraphs. By this the attacker may reconstruct the key.

Q13] In a Feistel cipher, each round is,

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, k_i)$$

the round function is simply,

$$F(R, k) = R \oplus k$$

$$\text{So, } R_i = L_{i-1} \oplus R_{i-1} \oplus k_i$$

Symbolic trace

$$\text{Round 1: } L_1 = R_0, \quad R_1 = L_0 \oplus R_0 \oplus K_1$$

$$\text{Round 2: } L_2 = L_0 \oplus R_0 \oplus K_1, \quad R_2 = L_1 \oplus R_1 \oplus K_2$$

$$= L_0 \oplus K_1 \oplus K_2$$

Numeric trace

$$\text{Given, } L_0 = \text{AAAAAAAA}$$

$$R_0 = \text{FOFOFOFO}$$

$$K_1 = \text{OFOFOFOF}$$

Round 1:

$$L_1 = R_0 = \text{FOFOFOFO} \leftarrow (\text{R}_0)$$

$$R_1 = L_0 \oplus R_0 \oplus K_1$$

$$= \text{AAAAAAAA} \oplus \text{FOFOFOFO} \oplus \text{OFOFOFOF}$$

$$= \text{AAAAAAAA} \oplus \text{FFFFFFFF}$$

$$= \text{66666666}$$

Round 2:

$$L_2 = R_1 = 55555555$$

$$K_2 = 33333333$$

$$R_2 = L_0 \oplus K_1 \oplus K_2$$

calculating $F(R_1, K_2) = R_1 \oplus K_2$

$$= 55555555 \oplus 33333333$$

here $55 = 01010101 = 66666666$

$$33 = 00110011$$

$$\text{XOR} \rightarrow 01100110 = 66$$

$$R_2 = L_1 \oplus F(R_1, K_2)$$

$$= F0F0F0F0 \oplus 66666666$$

$$= 96969696$$

$$F0 = 11110000$$

$$66 = 01100110$$

$$\text{XOR} \rightarrow 10010110 = 96$$

Final trace

$$(L_0, R_0) \rightarrow (L_1, R_1) \rightarrow (L_2, R_2)$$

$$(AAAAAAAA, F0F0F0F0)$$

$$\downarrow K_1 = 0F0F0F0F$$

$$(F0F0F0F0, 55555555)$$

$$\downarrow K_2 = 33333333$$

$$(55555555, 96969696)$$

$$L_1 = 55555555$$

$$R_2 = 96969696$$

141 The avalanche effect means that a small change in the input (such as flipping one bit) should cause a large and unpredictable change in the output.

Formally, if $E_k(P)$ is the encryption of plaintext P with key k , then for two plaintext P and P' differing in only one bit:

$$HD(E_k(P), E_k(P')) \approx \frac{n}{2}$$

where HD is the Hamming distance and n is the block size.

For an 8-bit block, ideally about: $\frac{8}{2} = 4$ output bits should change.

Toy 8-bit substitution-permutation Network:

Let the 8-bit input be divided into two 4-bit nibbles:

S-box

Input	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Output	E	4	D	1	2	F	0	8	3	A	6	C	5	9	0	7

Permutation: $P(b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8) = b_1 b_5 b_2 b_6 b_3 b_7 b_4 b_8$

Input 1

Let, $p = 12$

split 1/2

Substitution: $S(1) = 4, S(2) = D$

$12 \rightarrow 4D = 01001101$

After permutation, $\frac{11}{2} \rightarrow (19) \rightarrow (11) \rightarrow (11)$

$01001101 \rightarrow 01110001$

$C_1 = 71$

Flip one input bit

$p' = 13$

$12 = 00010010$

$13 = 00010011$

$S(1) = 4, S(3) = 1$

$13 \rightarrow 41 = 01000001$

After permutation,

$01000001 \rightarrow 00100001$

$C_2 = 21$

compare outputs.

$$C_1 = 01110001, C_2 = 00100001$$

$$\text{XOR: } 01110001 \oplus 00100001 = 01010000$$

different bits = 2 out of 8.

Q15 Brute-Force Resistance of Password vs PIN:

Given cracking Rate: $R = 10^9$ guesses/second.

8 character alphanumeric password:

Alphabet size = 62

Total possible password: $N = 62^8$

$= 2.18 \times 10^{14}$ combinations.

Worst case time: $T_{\text{worst}} = \frac{N}{R}$

$= 2.18 \times 10^5$ seconds

≈ 2.53 days.

Expected case time:

$T_{\text{expected}} = \frac{N}{2 \times R}$

$= \frac{2.18 \times 10^{14}}{2 \times 10^9}$

$= 1.09 \times 10^5$ seconds

≈ 1.26 days.

6 digit pin

each digit has 10 possibilities.

$N = 10^6 = 1,000,000 = 10^6$

worst case time:

$$T_{\text{worst}} = \frac{N}{10^9} = \frac{10^6}{10^9} = 10^{-3} \text{ seconds} = 1 \text{ millisecond}$$

expected - case time:

$$T_{\text{expected}} = \frac{N}{2 \times 10^9} = \frac{10^6}{2 \times 10^9} = 0.0005 \text{ s} = 0.5 \text{ ms}$$

Implications for password policy:

- ① A 6-digit PIN has very low brute force resistance if unlimited offline guesses are possible.
- ② longer password with a large character set provide much higher resistance.
- ③ passwords system should use long passwords/pass phrases, rate limiting, account lockout, and strong password hashing.
- ④ For sensitive systems MFA should also be used.

Longer password + larger alphabet + rate limiting = better security.

Q16] comparison of ECB, CBC, CFB, OFB and CTR mode.

1. ECB Diagram:

$P_1 \rightarrow \text{AES} \rightarrow C_1$

$P_2 \rightarrow \text{AES} \rightarrow C_2$

$P_3 \rightarrow \text{AES} \rightarrow C_3$

- parallel encryption and decryption is possible.
- Identical plaintext blocks produce identical ciphertext blocks
- Unique IV: No.

2. CBC

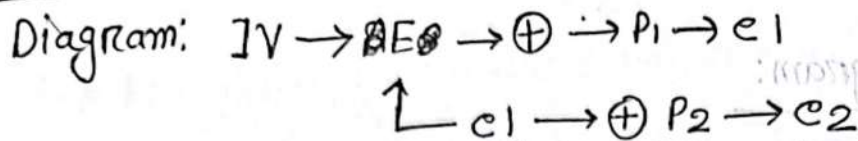
Diagram:

$\text{IV} \downarrow$
 $P_1 \oplus \rightarrow \text{AES} \rightarrow C_1$

$\uparrow$
 $P_2 \oplus \rightarrow \text{AES} \rightarrow C_2$
 $\uparrow$
 C_1

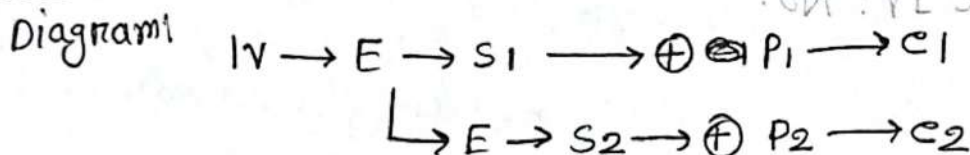
- parallel encryption: No
- parallel decryption: Yes
- Error: A bit error in c_i corrupts the current plaintext block and effect the bit of next block.
- Unique IV: No.

3. CFB



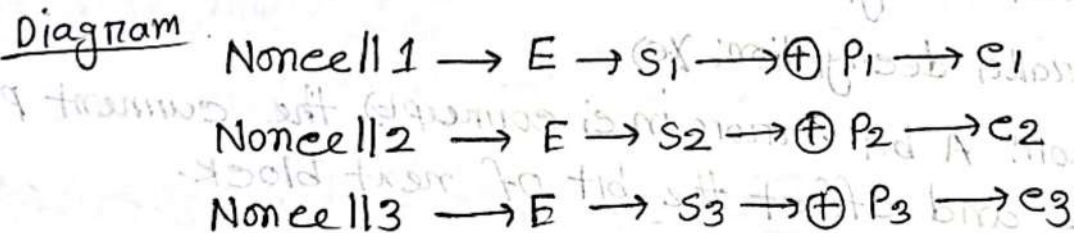
- Parallel encryption: No
- parallel decryption: Yes
- Error: A ciphertext bit error effect the current block and next block.
- Unique IV: Yes.

4. OFB



- Parallel encryption: No
- parallel decryption: No
- Error: A bit error effect only the corresponding bit.
- Unique IV: Yes and never reused.

5. CTR



- parallel encryption: Yes.
- parallel decryption: No
- Error: ~~A bit~~ effects only the plaintext bit.
- Unique IV: ~~no~~ unique and not reused.

(a) Live video stream over a lossy network:

CTR is suitable because:

- Encryption and decryption can be parallelized.
- A bit error affects only the corresponding plaintext block and doesn't propagate to other block.
- Suitable for continuous/streaming data.

(b) Static disk image:

CTR is suitable because:

- supports parallel encryption and decryption.
- No error propagation between blocks.
- Efficient for large amount of data.

After shift: [P, C, E, C]

P	C	E	C
00	00	00	00
00	00	00	00
00	00	00	00

Q17] Given state word: $[0x23, 0xA7, 0x4E, 0x19]$

Step 1: Subbytes

For each byte, using the 1st hex digit as row and 2nd hex digit as column.

0x23

Row 2, column = 3.

$$S(23) = D4$$

0xA7

Row = 4, column = 7

$$S(A7) = 63$$

0x4E

Row = 4, column = E

$$S(4E) = 2E$$

0x19

Row = 1, column = 9

$$S(19) = C6$$

After subbytes: $[D4, 63, 2E, C6]$

4x4 State matrix

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

Apply shift rows:

row 1: No shift

row 2: shift by 1

row 3: shift by 2

row 3: shift by 3.

Result:

D4	63	2E	26
00	00	00	00
00	00	00	00
00	00	00	00

Key expansion table:

for every group of 4 bytes

$T = W - 1$

for the first word of a round:

$T = g(w-1) = \text{row } g \oplus \text{subword } (w-1)$

where g consists of

(i) Rotword

(ii) Subword

(iii) $\text{row}[g] \oplus \text{row } w$

Example

Key = 000102030405060708090A0B0C0D0E0F

$W_0 = 00010203$

$W_1 = 04050607$

$W_2 = 08090A0B$

$W_3 = 0C0D0E0F$

Q18 AES-128 Rijndael Key schedule

For AES-128, the original key is 128 bits = 16 bytes
It's divided into four 32-bit words:

$$K = W_0 || W_1 || W_2 || W_3$$

Key expansion rule:

for every group of 4 words:

$$W_i = W_{i-4} \oplus T$$

For the first word of a round:

$$T = g(W_{i-1}) = \text{RotWord} \oplus \text{SubWord}(\text{RotWord}(W_{i-1}))$$

Where g consists of

- (i) RotWord
- (ii) SubWord
- (iii) $\text{Rcon}[j] / \text{Rcon XOR}$

Example

$$\text{Key} = 000102030405060708090A0B0C0D0E0F$$

$$W_0 = 00010203$$

$$W_1 = 04050607$$

$$W_2 = 08090A0B$$

$$W_3 = 0C0D0E0F$$

Round 1 (K₁)

$$W_3 = 0C0D0E0F \oplus 70700000 =$$

$$\text{Rotword}(W_3) \rightarrow 0D0E0F0E$$

Subword: 0 1 2 3 4 5 6 7 8 9 A B C D E F

$$S(0D) \rightarrow D7$$

$$S(0E) \rightarrow AB$$

$$S(0F) \rightarrow 7C$$

$$S(00) \rightarrow FE$$

D7AB76FE

$$\text{Rcon}_1 = 01000000$$

$$\text{XOR: } D7AB76FE \oplus 01000000 \\ = D6AB76FE$$

compute words

$$W_4 = W_0 \oplus D6AB76FE$$

$$= D6AB76FE$$

$$W_5 = W_1 \oplus W_4$$

$$= 04050607 \oplus D6AB76FE$$

$$= D2AF72FA$$

$$W_6 = W_2 \oplus W_5$$

$$= 08090A0B \oplus D2AF72FA = DAAG78F1$$

$$W_7 = W_3 \oplus W_6$$

$$= 0CDD0E0F \oplus DAAG78F1$$

$$= D6AB76FE$$

$$\therefore K_1 = D6AA74FD \quad D2AF72FA \quad DAAG78F1 \quad D6AB76FE$$

Round 2 (K2)

$$W_7 = D6AB76FE$$

$$\text{Rotword}(W_7) \rightarrow AB76FE D6$$

Subword:

$$AB \rightarrow 62$$

$$76 \rightarrow 38$$

$$FE \rightarrow BB$$

$$D6 \rightarrow FG$$

$$G238BBFG$$

$$\text{Rem}_2 = 02000000$$

$$\text{XOR} = G238BBFG \oplus 02000000$$

$$= 6038BBFG$$

$$W_8 = W_4 \oplus 6038BBFG$$

$$= D6AB76FE \oplus 6038BBFG$$

$$= B692CF0B$$

$$W_9 = W_5 \oplus W_8$$

$$= D2AF72FA \oplus B692CF0B$$

$$= 643DBDF1$$

$$W_{10} = \cancel{BE9B} W_6 \oplus W_9$$

$$= BE9BC500$$

$$W_{11} = W_7 \oplus W_{10}$$

$$= 6830B3FE$$

$$\therefore K_2 = B692CF0B \quad 643DBDF1 \quad BE9BC500 \quad 6830B3FE$$

a
b
c
d

01	01	03	02
10	03	02	01
03	02	01	10
02	10	01	03

b	⊕	a	⊕	030	⊕	020
b	⊕	030	⊕	020	⊕	a
030	⊕	020	⊕	a	⊕	a
020	⊕	a	⊕	a	⊕	030

Q19] In AES, Mixcolumns takes one column of 4 bytes and mixes them to produce a new column.

Each byte is treated as a polynomial in:

$$GF(2^8).$$

with the irreducible polynomial:

$$x^8 + x^4 + x^3 + x + 1.$$

In hexadecimal, this polynomial is 11B.

Fixed mixcolumns matrix:

For an input column,

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

AES uses:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Therefore:

$$\begin{bmatrix} 02a \oplus 03b \oplus c \oplus d \\ a \oplus 02b \oplus 03c \oplus d \\ a \oplus b \oplus 02c \oplus 03d \\ 03a \oplus b \oplus c \oplus 02d \end{bmatrix}$$

Example

Take the standard AES example:

DB
13
53
45

we calculate the first output byte:

$$Y_1 = (02 \times DB) \oplus (03 \times 13) \oplus 53 \oplus 45$$

Step 1 $(02 \times DB)$

multiplication by 02 means left shift by one bit.

$$DB = 11011011$$

$$\text{left shift} = 10110110$$

Since the original MSB is 1, XOR with 1B:

$$DB \times 02 = AD$$

Step 2 $(03 \times 13) = (02 \times 13) \oplus 13$

$$13 = 00010011$$

$$\text{left shift}_2 = \cancel{00010011} 00100110$$

MSB isn't 1.

$$\text{So, } 02 \times 13 = 26$$

$$\text{Therefore } 03 \times 13 = 26 \oplus 13$$

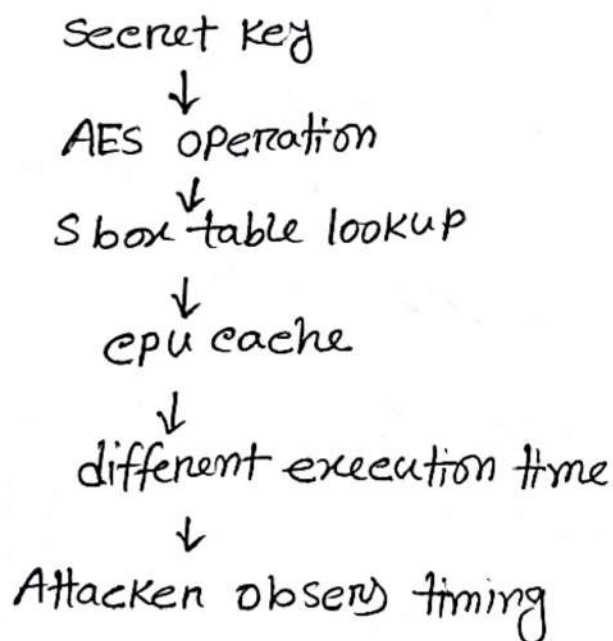
$$= 35$$

Q201

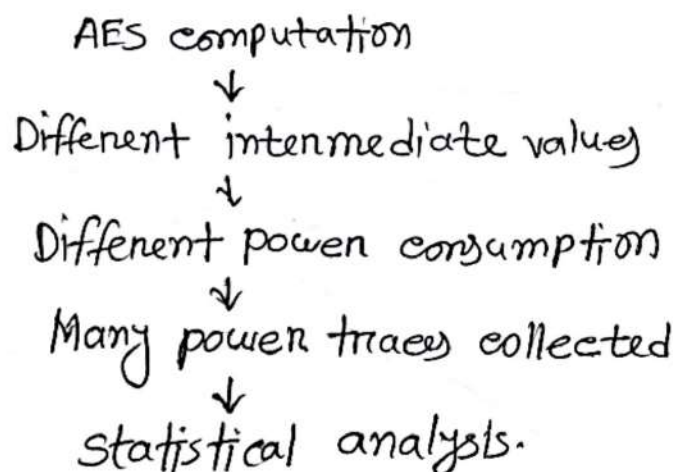
AES itself is mathematically secure against brute force attacks, but an implementation of AES can leak information through physical side channels such as execution time and power consumption.

1. Timing / Cache timing Attack:

A common software implementation uses a table based S-box.



2. Differential power analysis (DPA):



↓
Secret Key information

An attacker can collect many power traces and correlate them with predicted AES values to recover key information.

Countermeasures

(i) constant time implementation:

Use a constant time S box. so execution time and memory access don't depend on secret data.

- Trade off: many require more computation and can be slower than optimized lookup.

(ii) Masking

Randomly mask sensitive intermediate values.

$$\text{such as: } x' = x \oplus r$$

Where r = random mask.

- Trade off: Adds extra operation, randomness, generation and implementation complexity.

8.21 Key generation:

- (i) choose two large primes p, q ; $n = pq$
- (ii) compute Euler's totient: $\phi(n) = (p-1)(q-1)$
- (iii) choose e such that $1 < e < \phi(n)$, $\gcd(e, \phi(n)) = 1$
- (iv) compute d such that $ed = 1 \pmod{\phi(n)}$

Thus public key = (e, n)

private key = (d, n)

(for all messages M)

Encryption: $C = M^e \pmod{n}$

Decryption: $M' = C^d \pmod{n}$

Deriving correctness from Euler's Theorem:

Euler's Theorem: if $\gcd(M, n) = 1$ then $M^{\phi(n)} \equiv 1 \pmod{n}$.

Since $ed = 1 + k\phi(n)$,

$$C^d = (M^e)^d = M^{ed} = M^{1+k\phi(n)} =$$

Case 1 $\gcd(M, n) = 1$

By Euler's theorem,

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

$$M^{ed} = M^{1+k\phi(n)} = M(M^{\phi(n)})^k = M \cdot 1^k$$

$$M^{ed} \equiv M \pmod{n}.$$

case 2: $\gcd(M, n) \neq 1$.

since $n = pq$, suppose $p \mid M$ then,

$$M \equiv 0 \pmod{p}$$

Mod p since $M \equiv 0 \pmod{p}$

$$M^e \equiv 0^e \equiv 0 \equiv M \pmod{p}$$

Mod q since $\gcd(M, q) = 1$, Fermat's little theorem

$$\text{gives } M^{q-1} \equiv 1 \pmod{q}$$

$$\text{since } (q-1) \mid \phi(n), \quad e-1 = k\phi(n) = k(p-1)(q-1)$$

$$\text{so, } (q-1) \mid (e-1)$$

$$M^e = M \cdot M^{e-1}$$

$$= M (M^{q-1})^{k(p-1)} = M \cdot 1 = M \pmod{q}$$

By CRT

$$M^e \equiv M \pmod{pq} = M \pmod{n}$$

for all M , including $\gcd(M, n) \neq 1$.

8221

given, $p=11$, $q=13$, $H(M)=23$.1. Key generation

$$n = pq = 11 \times 13 = 143$$

$$\phi(n) = (11-1)(13-1) = 10 \times 12 = 120$$

$$\text{let, } e=7$$

because $\gcd(e, 120) = 1$ with $1 < e < 120$.find d

$$ed \equiv 1 \pmod{120}$$

$$7d \equiv 1 \pmod{120}$$

$$\Rightarrow 120 = 17(7) + 1$$

$$\Rightarrow 1 = 120 - 17(7)$$

$$\Rightarrow -17(7) = 1 \pmod{120}$$

$$\text{Since } 120 \equiv 0 \pmod{120}$$

$$-17(7) = 1 \pmod{120}$$

$$\text{So, } d = -17 = -17 + 120 = 103$$

$$\therefore d = 103$$

Therefore public key $E = (e, n) = (7, 143)$

private key $D = (d, n) = (103, 143)$

2. Signing

$$S = H(M)^d \pmod{n} = 23^{103} \pmod{143}$$

Binary exponent of 103 = $164 + 32 + 4 + 2 + 1$
 $= (1100111)_2$

using repeated squaring:

$$23^1 = 23$$

$$23^2 = 529 = 3(143) + 100 = 100$$

$$23^4 = (100)^2 = 10000 = 69(143) + 133 = 133$$

$$23^8 = (133)^2 = 17689 = 123(143) + 100 = 100$$

$$23^{16} = (100)^2 = 133$$

$$23^{32} = 133^2 = 100$$

$$23^{64} = 133$$

$$23^{103} = 23^{64} \times 23^{32} \times 23^4 \times 23^2 \times 23^1 \pmod{143}$$

$$= 133 \times 100 \times 133 \times 100 \times 23 \pmod{143}$$

23) Derive a

let payload size = P bytes

ESP header = H bytes

Authentication data = A bytes

padding = P bytes

In tunnel mode, the ESP overhead is:

$$O = H + A + Pad$$

Therefore overhead percentage is

$$\text{Overhead \%} = \frac{H + A + Pad}{P + H + A + Pad} \times 100$$

Given,

$$P = 1200, H = 8, A = 16, Pad = 2$$

$$\text{Overhead} = H + A + Pad = 8 + 16 + 2 = 26 \text{ bytes}$$

$$\text{Total packet size} = P + H + A + Pad = 1200 + 26$$

$$= 1226 \text{ bytes}$$

$$\text{overhead (\%)} = \frac{H + A + Pad}{P + H + A + Pad} \times 100 = \frac{26}{1226} \times 100 = 2.12\%$$

Scaling:

Small IoT packets: Overhead is a larger percentage because the fixed ESP overhead is large compared with small payload.

12-8

Large payloads: the same fixed overhead becomes a smaller percentage.

$$\text{Overhead \%} = \frac{C}{P} \times 100 \quad (\text{for large } P, \text{ where } C = \text{fixed overhead})$$

inversely proportional to payload size.

So, small payload $\rightarrow$ high overhead %.

Large payload $\rightarrow$ low overhead %.

Q24] Diffie-Hellman Key exchange:

$$P = 23, g = 5.$$

Aleya's private key: $a = 6$

Badol's private key: $b = 15$

Aleya's private-public key:

$$A = g^a \mod P = 5^6 \mod 23$$

$$5^2 = 25 \equiv 2 \mod (23)$$

$$5^4 = (2)^2 = 4$$

$$5^6 = 5^4 \cdot 5^2 = 4 \times 2 = 8$$

$$A = 8.$$

Badol's public key:

$$B = g^b \mod P = 5^{15} \mod 23$$

$$5^1 = 5$$

$$5^2 = 25 \equiv 2$$

$$5^4 = 2^2 \equiv 4$$

$$5^8 = 4^2 \equiv 16$$

$$\therefore 5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5^1 = 16 \times 4 \times 2 \times 5 \mod (23)$$

$$= 640 \mod (23)$$

$$= 19$$

$$B = 19.$$

Aleja and Badol exchange $A=8$ and $B=19$ over the public channel.

Shared Secret

Aleja calculates: $K = B^A \text{ mod } P = 19^8 \text{ mod } 23$

$$19^1 \equiv -4 \pmod{23}$$

$$19^2 \equiv (-4)^2 \equiv 16$$

$$19^4 \equiv (16)^2 \equiv 256 \pmod{23} \equiv 3$$

$$\therefore 19^8 \equiv 10^4 \cdot 19^4 \equiv 3 \times 16 \equiv 48 \equiv 2$$

$$\therefore \text{Saleja} = 2.$$

Badol calculates: $K = A^B \text{ mod } P$ is of value 2

$$8^{16} \pmod{23}$$

$$8^1 \equiv 8$$

$$8^2 \equiv 64 \equiv 18$$

$$8^4 \equiv (18)^2 \equiv 2$$

$$8^8 \equiv 2^2 \equiv 4$$

$$\therefore 8^{16} \equiv 8^8 \cdot 8^4 \cdot 8^2 \cdot 8 \equiv 4 \times 2 \times 18 \times 8 \equiv 1152$$

$$1152 \text{ mod } 23 = 2.$$

$$\therefore K_{\text{badol}} = 2$$

$$\therefore K = 2$$

So, both parties derive the same shared secret.

Man in the Middle Attack

Basic DH doesn't authenticate the public values A and B .
An attacker can intercept them and replace them with their own public values:

$A \rightarrow A_{\text{attacker}}$; $B \rightarrow B_{\text{attacker}}$

Then the attacker establishes secret key with A and B. The attacker can therefore read, modify and forward messages between them.

Authenticated DH / STS

The station to station protocol adds authentication using digital (signatures) and certificates.

The exchanged DH values are authenticated by signing the relevant DH information.

Therefore an attacker can't successfully replace the public keys without failing authentication.

DH + Authentication $\rightarrow$ prevents MITM.

Q26] computational cost:

Method	Security Level	Main computation	Cost
RSA Key Transport	3072-bit RSA	Modular exponentiation with 3072-bit numbers	Relatively high
DH	3072-bit DH	Modular exponentiation with 3072-bit numbers	Relatively high
ECDH	256-bit ECDH	Elliptic-curve point multiplication	Much lower

Forward Secrecy:

In RSA key transport, the client encrypts the session key using the server's long-term RSA public key. If an attacker records encrypted traffic and later obtains the server's private RSA key, old session keys can potentially be recovered.

In (EC) DHE, fresh temporary private/public keys are generated for each session.

New ephemeral key per session $\rightarrow$ Forward Secrecy.

Even if the long-term authentication key is compromised later, previously recorded sessions remain protected.

So, Modern TLS prefers (EC) DHE because it provides forward secrecy.

Q26] In an ElGamal scheme with $p=467$, $g=2$, private key $x=127$, message has $H(M)=100$, and nonce $k=213$ (with $\gcd(k, p-1)=1$): compute the public key, generate the signature (r, s) , and verify it.

$\Rightarrow$ given, $p=467$, $g=2$, $x=127$, $H(M)=100$, $k=213$.

ElGamal public key: $y = g^x \bmod p$
 $= 2^{127} \bmod 467$

here $127 = 64 + 32 + 16 + 8 + 4 + 2 + 1$.

$$2^1 = 2$$

$$2^2 = 2^2 = 4$$

$$2^4 = 16$$

$$2^8 = 256$$

$$2^{16} = (256)^2 = 65536 \bmod 467 \equiv 156$$

$$2^{32} = (156)^2 = 24336 \equiv 62$$

$$2^{64} = (62)^2 = 3844 = 369$$

$$2^{127} = 369 \times 62 \times 156 \times 256 \times 16 \times 4 \times 2 \pmod{467}$$

$$\therefore y \equiv 132$$

$$\therefore \text{public Key } (p, g, y) = (467, 2, 132).$$

generate signature (r, s)

$$r = g^k \text{ mod } p$$

$$= 2^{213} \text{ mod } 467 = 29$$

compute $k^{-1} \text{ (mod } p-1)$:

$$213^{-1} \text{ (mod } 466)$$

euclidean division:

$$466 = 2(213) + 40$$

$$213 = 5(40) + 13$$

$$40 = 3(13) + 1$$

back substitute:

$$1 = 40 - 3(13)$$

$$= 40 - 3(213 - 5 \times 40)$$

$$= 16(40) - 3(213)$$

$$= 16(466 - 2 \times 213) - 3(213)$$

$$= 16(466) - 35(213)$$

$$\therefore -35(213) \equiv 1 \text{ (mod } 466)$$

$$213^{-1} = 431 \text{ (mod } 466)$$

$$s = k^{-1} (H(M) - xr) \text{ (mod } p-1)$$

$$= 431 (100 - 127 \times 29) \text{ (mod } 466)$$

$$= 431 \times (-3583) \text{ mod } 466 \equiv 51 \quad (r, s) = (29, 51)$$

Verification

$$g^{H(M)} \pmod{p} = y^n \pmod{p}$$

$$L.H.S = 2^{200} \pmod{467} = 189$$

$$R.H.S = 132^{29} \times 29^{61} \pmod{467} = 189$$

$\therefore$ Signature is valid.

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

$$10 + 3(13) = 50$$

Q-27

Q27] prove algebraically that ElGamal signature verification equation $g^{H(M)} = y^n r^s \pmod{p}$ holds for a correctly generated signature, starting from the definition $s = k^{-1}(H(M) - xn) \pmod{p-1}$.

$$\Rightarrow \text{Given, } y = g^x \pmod{p}$$

$$\text{the signature equation, } s = k^{-1}(H(M) - xn) \pmod{p-1}$$

Step 1

$$s = k^{-1}(H(M) - xn) \pmod{p-1}$$

multiply both sides by k ,

$$ks \equiv H(M) - xn \pmod{p-1}$$

$$\text{Therefore, } H(M) = ks + xn \pmod{p-1}$$

$$\text{Now, } g^{H(M)} = g^{xn + ks} \pmod{p}$$

$$= g^{xn} \cdot g^{ks}$$

$$= (g^x)^n \cdot (g^k)^s \pmod{p}$$

$$\text{Using, } y = g^x \pmod{p} \text{ and } r = g^k \pmod{p}.$$

$$g^{H(M)} = y^n \cdot r^s \pmod{p}$$

$$\text{Hence } g^{H(M)} \equiv y^n \cdot r^s \pmod{p}.$$

[8-28]

Q28] on the elliptic curve $y^2 = x^3 + x + 1 \pmod{23}$: verify that $P(3, 10)$ lies on the curve, compute $2P$ via point doubling, and compute $P+Q$ for $Q = (9, 7)$, showing the full modular-arithmetic derivation of the slope in each case.

⇒ given the elliptic curve: $y^2 = x^3 + x + 1 \pmod{23}$
verify $P(3, 10)$,

$$L.H.S = y^2 = 10^2 = 100 \pmod{23} \equiv 8 \pmod{23}$$

$$R.H.S = 3^3 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

So, $P(3, 10)$ lies on the curve.

Compute $2P$ (point doubling)

For doubling, $\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}$ where $a=1, x_1=3, y_1=10$

$$= \frac{3 \cdot 3^2 + 1}{2 \cdot 10} \pmod{23}$$

$$= \frac{28}{20} \pmod{23}$$

$$\text{here } 28 \equiv 5 \pmod{23}$$

$$20^{-1} \equiv 15 \pmod{23}$$

$$\text{because } 20 \times 15 = 300 \equiv 1 \pmod{23}$$

$$\therefore \lambda = 5 \times 15 = 75 \equiv 6 \pmod{23}$$

New coordinates:

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 2 \cdot 3 = 36 - 30 \equiv 7$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$= 6(3-7) - 10 = -34 \equiv 12 \pmod{23}$$

Therefore $2P = (7, 12)$

compute $P+Q$

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}$$

$$= \frac{7 - 10}{3 - 7} = \frac{-3}{-4}$$

$$-3 \equiv 20 \pmod{23}$$

$$6^{-1} \pmod{23} : 6 \times 4 = 24 \equiv 1 \pmod{23}$$

$$\therefore 6^{-1} \equiv 4 \pmod{23}$$

$$\therefore \lambda = 20 \times 4 = 80 \equiv 11 \pmod{23}$$

Now, $x_3 = \lambda x_1 - x_2$

$$= (11) \times 3 - 7 = 10 \equiv 17 \pmod{23}$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$= 11(3-17) - 10 = -154 - 10 = -164$$

$$-164 \equiv -164 + 184 = 20 \pmod{23}$$

$$\therefore P+Q = (17, 20)$$

Ans

Q29] on the curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$ with base point $G = (2, 5)$ of order $n = 19$ and private key $d = 9$: compute the public key $Q = dG$, generate an ECDSA signature (r, s) for $H(M) = 8$ using nonce $k = 3$, and verify it against Q .

$\Rightarrow$ Given, $y^2 \equiv x^3 + 7x + 10 \pmod{37}$

$G(2, 5)$, $n = 19$, $d = 9$, $H(M) = 8$, $k = 3$.

For $G(2, 5)$:

$$y^2 = 25$$

$$x^3 + 7x + 10 = 2^3 + 7 \cdot 2 + 10 = 32 \equiv 1 \pmod{37}$$

$$25 \not\equiv 32 \pmod{37}$$

So, $G(2, 5)$ doesn't lie on the given curve.

Therefore, the problem is internally inconsistent, and a valid ECDSA signature can't be correctly verified.

Step 1: public Key

$$Q = dG = 9G$$

slope

Formula

Deriv

Doubling: $\lambda = \frac{3x_1^2 + a}{2y_1}$, $(\mathbb{F}_p) \text{ mod } p$

Addition: $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$, $\frac{y_2 - y_1}{x_2 - x_1} = \frac{19 - 6}{10 - 3} = \frac{13}{7}$

$$x_3 = \lambda^2 - x_1 - x_2$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

2G $\lambda = \frac{3 \cdot 2^2 + 7}{2 \cdot 6} = \frac{19}{10} = 19 \cdot 10^{-1} \pmod{37}$ $a=7$
 $= 19 \cdot 26 = 13 \pmod{37}$ $\phi(37)$

$$x_3 = 13^2 - 4 = 165 \pmod{37} \equiv 17$$

$$y_3 = 13(2 - 17) - 6 = -200 \pmod{37} \equiv 22$$

$$2G \rightarrow (17, 22)$$

4G $\rightarrow 2 \cdot 2G$: $\lambda = \frac{3 \cdot 17^2 + 7}{2 \cdot 22} = \frac{23}{7} = 23 \cdot 7^{-1}$

here $7^{-1} = 16$

$$\lambda = 23 \times 16 = 368 \equiv 35$$

$$x_3 = 35^2 - 17 - 17 = 1191 \equiv 7$$

$$y_3 = 35(17 - 7) - 22 = 328 \equiv 32$$

$$4G \rightarrow (7, 32)$$

$$8G: 2.4G(7, 32)$$

$$\lambda = \frac{3 \cdot 7 + 7}{2 \cdot 32} = \frac{154}{64} = \frac{6}{27} \equiv 6 \cdot 27^{-1} = 6 \cdot 11 = 66 \equiv 29 \pmod{33}$$

$$x_3 = 29^2 - 7 - 7 = 827 \equiv 13$$

$$y_3 = 29(7 - 13) - 32 = 29(-6) - 32 = -206 \equiv 16$$

$$8G = (13, 16)$$

$$9G \rightarrow 8G + G$$

$$\lambda = \frac{6 - 16}{2 - 13} = 1$$

$$x_3 = 1^2 - 13 - 2 = -14 \equiv 23$$

$$y_3 = 1(13 - 23) - 16 = -26 \equiv 11$$

public key: $9G(23, 11)$

Signature

$$KG = 3G = 2G + G$$

$$\lambda = \frac{6 - 22}{2 - 17} = \frac{17}{15}$$

$$= 17 \cdot 6 \equiv 11$$

$$x_2 = 121 - 19 \equiv 28 \pmod{19}$$

$$y_3 = 11(17 - 28) - 22 \equiv 6 \pmod{19}$$

$$3G(28, 6) = (28, 6)$$

$$\text{Thus } r = x_1 g \pmod{n}$$

$$= 28 \pmod{19}$$

$$= 9$$

$$k^{-1} = 3^{-1} \pmod{19} = 13 \quad \text{Since } 3 \times 13 = 39 \equiv 1 \pmod{19} \therefore$$

$$\text{Now, } s = k^{-1} [H(M) + dr] \pmod{n}$$

$$= 13 [8 + 9(9)] \pmod{19}$$

$$= 13 \cdot 89 \pmod{19}$$

$$\equiv 17$$

$$\text{So, signature } (r, s) = (9, 17)$$

Verification

$$w = s^{-1} \pmod{19} = 17^{-1} \pmod{19} = 9$$

$$u_1 = H(M) w = 8(9) \equiv 16$$

$$u_2 = rw = 9 \times 9 = 81 \equiv 5$$

$$\begin{aligned} \text{calculate } X &= u_1 a + u_2 b \pmod{p} \\ &= 16a + 3b \pmod{19} \\ &= (22, 36) \cdot (2, 12) \pmod{19} \end{aligned}$$

Therefore,

$$v = x \pmod{19} = 22 \pmod{19} = 3$$

$$3 \neq r = 9$$

$\therefore$ signature doesn't verify.

$$r \pmod{p} = [ab + (M)H] \pmod{p} = 2 \pmod{19}$$

$$s \pmod{p} = [e + a(e)] \pmod{p} = 13 \pmod{19}$$

$$(s \pmod{p}) \cdot s = 13 \cdot 3 = 39$$

$$\cdot H = 39 \cdot 12 = 468$$

$$(r, s) = (2, 3) \text{ not a valid signature}$$

Verification

$$w = a \pmod{p} \cdot v \pmod{p} = 2 \pmod{19} \cdot 3 \pmod{19} = 6 \pmod{19}$$

$$w \cdot H = 6 \cdot 12 = 72 \pmod{19} = 10$$

$$w \cdot H = 6 \cdot 12 = 72 \pmod{19} = 10$$

Q30] compare the key-size security equivalence between RSA, @. ElGamal and EEC (e.g, 256 bit EEC $\approx$ 3072 bit RSA / ElGamal at the $\sim$ 128 bit security level), and explain why EEC is preferred for resource constrained devices such as smart cards and IoT sensors.

$\Rightarrow$ RSA vs ElGamal vs EEC key size:

At approximate 128-bit security, the commonly used key-size equivalence is:

Algorithm	Approx. Key size	Security Level
- RSA	3072-bit	$\approx$ 128-bit
ElGamal	3072-bit	$\approx$ 128-bit
EEC	128 256-bit	$\approx$ 128-bit

Thus:

256-bit EEC $\approx$ 3072 bit RSA / ElGamal $\approx$ 128-bit Security

EEC is preferred for IoT and smart cards:

① smaller keys: EEC provides the same security with much smaller keys.

② Less memory: Smaller keys and certificates require less storage.

③ Lower bandwidth: smaller public keys/signatures reduce communication overhead.

④ Lower computational and power cost: This is important for battery-powered IoT sensors and smart cards.

Ecc = similar security + smaller keys + lower resource

Algorithm	Approx. Key Size (bits)	Approx. Signature Size (bits)
Ecc	~256	~350
RSA	~3072	~2048

Therefore Ecc is well suited for resource-constrained devices such as smart cards and IoT sensors.

① Smaller keys: Ecc provides the same security with much smaller keys.

Q31] Formally define preimage resistance, second-preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday attack complexity ($\approx 2^{n/2}$) for finding a collision in an n -bit hash, and explain why this motivates 256-bit (rather than 128-bit) output lengths for modern hash function.

⇒ Hash function security properties:

Let, $H: \{0,1\}^* \rightarrow \{0,1\}^n$ be an n -bit cryptographic hash function.

1. preimage resistance

Given a hash function value h , it should be computationally infeasible to find any message M such that,

$$H(M) = h$$

expected work: $\approx 2^{n-1}$ (worst case 2^n).

2. Second preimage resistance

Given a message M_1 , it should be infeasible to find a different message M_2 such that

$$M_2 \neq M_1, H(M_2) = H(M_1)$$

expected work: $\approx 2^{n-1}$

3. collision resistance:

It should be infeasible to find any two different messages M_1, M_2 such that,

$$M_1 \neq M_2 \quad H(M_1) = H(M_2)$$

unlike preimage attacks, collision attack requires only about $2^{n/2}$ operations.

Birthday attack derivation

There are 2^n possible hash outputs.

After hashing K random messages, the approximate probability of a collision is:

$$P \approx \frac{K(K-1)}{2 \cdot 2^n}$$

A collision becomes likely when $P \approx 1$:

$$K(K-1) \approx 2 \cdot 2^n \Rightarrow K^2 \approx 2^n$$

$$\therefore K \approx 2^{n/2}$$

This is called birthday attack.

Why 256-bit is preferred over 128-bit

For a 128-bit hash: $2^{128/2} = 2^{64} \rightarrow$ collision work is approximately 2^{64}

256-bit hash: $2^{256/2} = 2^{128} \rightarrow$ collision work is 2^{128} .

Therefore modern systems use 256 bit hash output for stronger security.

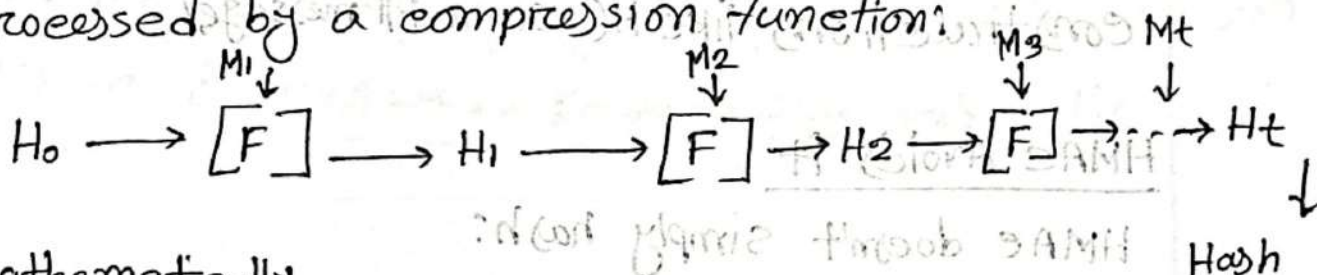
Q32] Trace the Merkle-Damgård construction with a diagram showing message blocks, compression function blocks and chaining variables. Explain the length extension Attack that this construction is vulnerable to, and how HMAC and SHA-3 avoid it.

⇒ 1. Merkle-Damgård construction

The message is divided into fixed size blocks:

$$M = M_1 \parallel M_2 \parallel M_3 \dots \parallel M_t$$

Starting with an initial value H_0 , each block is processed by a compression function:



Mathematically,

$$H_i = F(H_{i-1}, M_i)$$

and the final hash is $H(M) = H_t$

2. Length extension Attack

In Merkle-Damgård hashes, the final hash H_t acts as the chaining value.

Suppose an attacker knows $H(M)$ but doesn't know M .

The attacker can use this hash as the starting state and process additional data x .

$$H(M \parallel \text{padding} \parallel x)$$

Thus the attacker computes the hash of an extended message without knowing the original message.

$$H(M) \rightarrow H(M \parallel \text{padding} \parallel x).$$

This is called length extension attack and affects hashes such as SHA-256 when used naively in constructions like $H(\text{secret} \parallel \text{message})$.

HMAC Avoids It

HMAC doesn't simply hash:

$$H(\text{secret} \parallel \text{message})$$

Instead, it uses nested hashing:

$$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

The secret, key and nested structure prevent an attacker from directly continuing the internal hash.

SHA-3

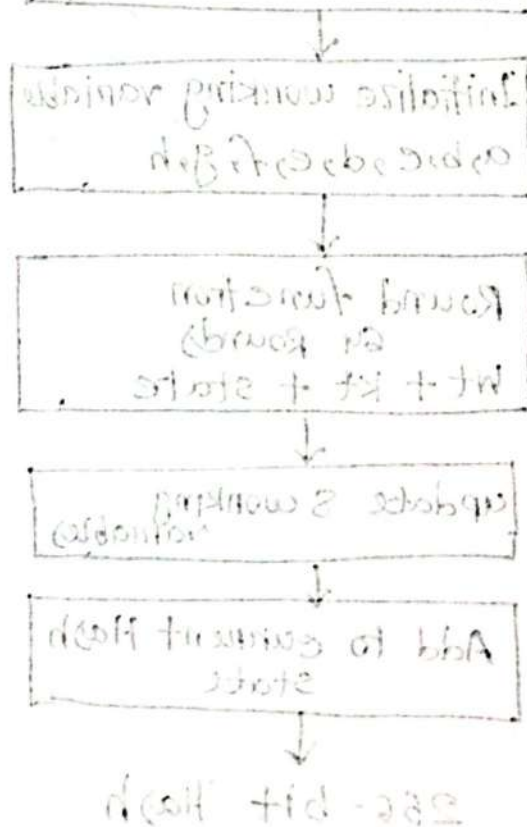
SHA-3 uses a sponge construction, not merkle

- Damgard

Message $\rightarrow$ Absorb $\rightarrow$ permutation $\rightarrow$ absorb $\rightarrow$... $\rightarrow$ squeeze $\rightarrow$ Hash.

Its internal state isn't exposed as a simple final chaining value, so the traditional Merkle-Damgard length-extension attack doesn't apply.

So, Merkle-Damgard $\rightarrow$ vulnerable to length-extension
 HMAC & SHA-3 $\rightarrow$ Remove this vulnerability.

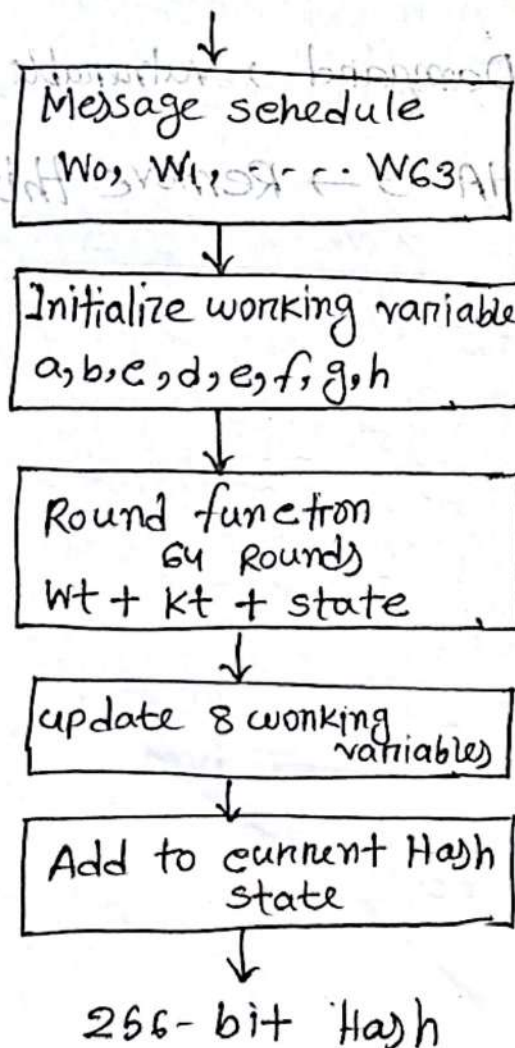


Q33] Explain the high level internal structure of the SHA-256 compression function - message schedule expansion, the eight working variables, and the round function - using a labelled block diagram.

⇒ SHA-256 processes each 512-bit message block using a compression function with 64 rounds.

Block diagram

512-bit message block



Main stages

① Message schedule:

The 512-bit block is divided into sixteen 32-bit words

$W_0, W_1, \dots, W_{15}$

These are expanded into 64 words:

$W_0, W_1, \dots, W_{63}$

This provides different input for each round.

② Eight working variables:

a, b, c, d, e, f, g, h .

These are initialized from the current 256-bit hash state and hold intermediate values during rounds.

③ Round Function:

SHA-256 performs 64 rounds. Each round uses:

- one message schedule word W_t
- one round constant K_t
- The eight working variables

The round function mixes these values using bitwise operations and updates a, b, c, d, e, f, g, h .

④ Final Hash update:

After 64 rounds, the resulting 8 variables are added to the previous hash state to produce the new 256-bit chaining value. ~~This~~ 512-bit block $\xrightarrow{64 \text{ rounds}}$ 256-bit state.

8341

Formally define the shortest vector problem (SVP) and the shortest Independent vectors problem (SIVP) in lattice theory, and explain why both are believed to remain hard even for quantum computers.

⇒ Let L be a Lattice generated by linearly independent basis vectors.

$$B = \{b_1, b_2, b_3, \dots, b_n\}$$

A lattice is,

$$L(B) = \left\{ \sum_{i=1}^n z_i b_i \mid z_i \in \mathbb{Z} \right\}$$

Shortest vector problem (SVP)

SVP asks us to find the non-zero shortest vector in a lattice.

formally given a lattice L , find

$$v \in L \setminus \{0\}$$

such that, $\|v\| = \lambda_1(L(B))$

$$\text{Where } \lambda_1(L(B)) = \min_{v \in L(B) \setminus \{0\}} \|v\|$$

SIVP

SIVP asks us to find n linearly independent lattice vectors that are as short as possible.

Formally, find linearly independent

$$v_1, v_2, v_3, \dots, v_n \in L(B)$$

such that,

$$\max \|v_i\| \leq \lambda_n(L(B))$$

Where λ_n is the smallest radius containing n linearly independent lattice vectors.

SVP and SIVP are believed to be hard even for quantum computers because:

- No efficient quantum ~~computer~~ algorithm is currently known for solving general high-dimensional SVP and SIVP.
- Shor's algorithm efficiently solves integer factorization and discrete logarithms, but it doesn't solve general lattice problems.

- Therefore lattice based cryptography uses the presumed hardness of these problems for post quantum security

SVP/SIVP: no known efficient quantum solution

Shor's algo: factorization/DLP, not general SVP, SIVP.

Q36]

Explain the core idea behindCRYSTALS - Kyber's Modulo-Learning-with-Errors (Modulo-LWE) Key encapsulation mechanism, including how "noise" is used to hide the secret, and why this construction resists both classical and quantum attacks.

⇒ CRYSTALS Kyber is a lattice based post quantum Key encapsulation mechanism (KEM).

It's security is based on the hardness of Module-LWE.

core idea

Kyber uses a noisy equation

$$b = As + e$$

Where, A = public random matrix

s = secret vector

e = small random noise/error

b = public value

Because of the noise e , the attacker can't easily recover s from A and b .

Key encapsulation

- Key generation: Generate secret s and noise e , then compute public key from A, s, e .
- Encapsulation: The sender uses the public key and noise to create a ciphertext and shared secret. Ciphertext (u, v) is sent to the receiver.
- Decapsulation: The receiver uses the secret key to recover the same shared secret.

The noise is essential because it makes the underlying equations difficult to solve.

Why post quantum secure?

Module-LWE is related to difficult lattice problems. No efficient classical or quantum algorithm is currently known for solving these problems at Kyber's security parameters.

Unlike RSA and ECC:

Shor's algorithm doesn't efficiently break Module-LWE.

Therefore,

Kyber's provide protection against known classical and quantum attacks.

8361

Kerberos Authentication protocol:

Kerberos uses a trusted third party to authenticate a client without sending the password repeatedly.

It has 3 main stages:

1. Authentication Server (AS) exchange.
2. Ticket granting server (TGS) exchange.
3. client server (CS) exchange.

1. AS exchange

- client sends ID_c , ID_{tgt} to the AS.
- AS verifies the client and creates a ^{temporary} session key $K_{c,tgs}$.
- AS sends:
 - $K_{c,tgs}$, encrypted with the client's secret key K_c .
 - T_{gt} , encrypted with the TGS's secret key.
- T_{gt} contains information such as client ID, $K_{c,tgs}$ and its lifetime.

2. TGS exchange

- client sends the T_{gt} + Authenticator to TGS and request access to service s .
- TGS verifies and creates a new session key $K_{c,s}$.
- TGS sends:
 - $K_{c,s}$, encrypted using $K_{c,tgs}$
 - Service ticket $T_{c,s}$, encrypted with K_s .

3. Client-server exchange

- client sends $T_{c,s}$ + Authenticator to server s .
- Server decrypts the ticket using K_s and obtains $K_{c,s}$.
- Server verifies the authenticator and sends back $Timestamp + 1$ encrypted with $K_{c,s}$. Thus server is genuine.

Q37 |

plain hash-based MAC

A simple MAC can be constructed as:

$$MAC = H(K || M)$$

Where K is the secret key and M is message.

If H is a merkle-Damgard Hash such that SHA-256, the final hash value is also the final internal chaining state.

Therefore If an attacker knows $H(K || M)$.

They can use it as the starting state and continue hashing an additional message X :

$$H(K || M || \text{padding} || X)$$

Without knowing K .

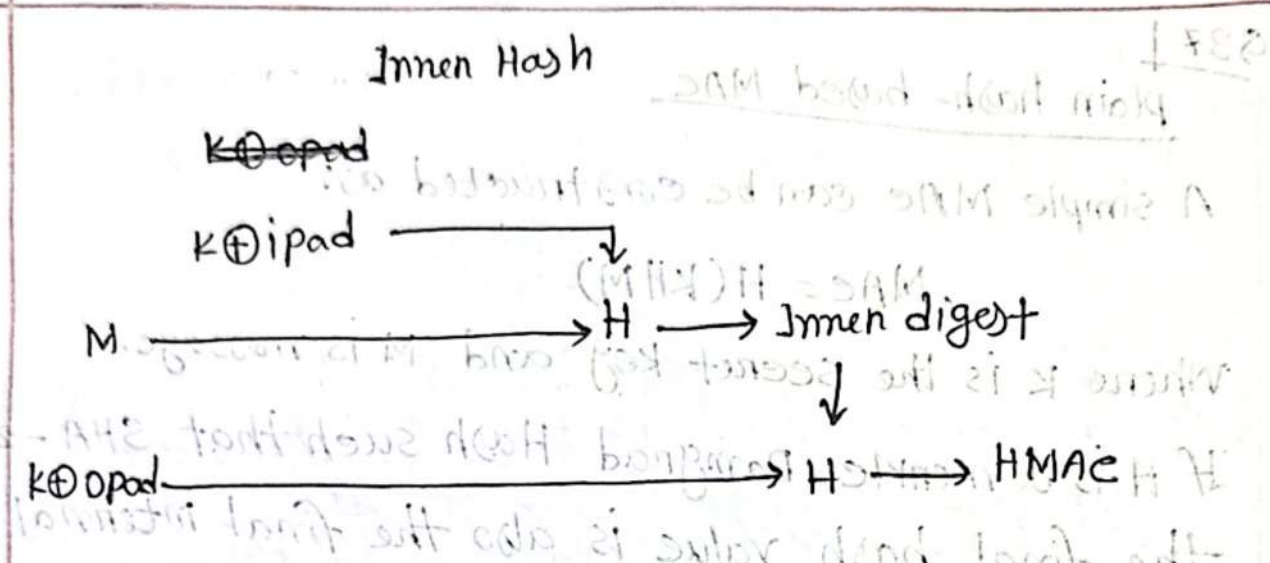
This is called length-extension attack.

HMAC avoids it

HMAC is constructed as,

$$HMAC(K, M) = H((K \oplus \text{opad}) || H((K \oplus \text{ipad}) || M))$$

It has two hash layers:



The attacker only sees the outer hash output.

To extend the message, the attacker would need to modify the inner hash:

$$H((K \oplus \text{ipad}) \parallel M \parallel \text{padding} \parallel x)$$

but the inner hash digest is already compressed inside the outer hash. The attacker can't simply use the final HMAC value as the inner hash state.

Hence, the classic length-extension attack doesn't work against HMAC.

$$(M \parallel (\text{padding} \oplus x)) \parallel ((\text{padding} \oplus x)) H = (M \parallel x) \text{ HMAC}$$

Toy-Merkle-Damgard hash:

initial state, $IV = 10$.

$$e(s, b) = (3s + b) \bmod 100$$

Let, $K = 7$, $M(12, 20)$

plain MAC

$$H(K || M)$$

$$e(10, 7) = (3 \times 10 + 7) \bmod 100 = 37 \bmod 100 = 37$$

$$e(37, 12) = (3 \times 37 + 12) \bmod 100 = 123 \bmod 100 = 23$$

$$e(23, 20) = (23 \times 3 + 20) \bmod 100 = 89$$

$$MAC = 89$$

If the attacker wants to append $x=6$, they can continue from the state 89.

$$e(89, 6) = 72$$

Thus attacker constructed the MAC of extended msg.

$$H(K || M || \text{padding} || x) = 72$$

without knowing K .

HMAC example

Let $ipad = 54$, $opad = 92$.

$$K \oplus ipad = 7 \oplus 54 = 49$$

$$K \oplus opad = 7 \oplus 92 = 91$$

inner hash:

$$c(10, 49) = (10 \times 3 + 49) \mod 100 = 79$$

$$c(79, 12) = 49$$

$$c(49, 20) = 67$$

inner digest = 67.

Outer hash:

$$c(10, 91) = 21$$

$$c(21, 67) = 30$$

HMAC = 30.

For extended message $M || 5$:

$$c(67, 5) = 6$$

New HMAC

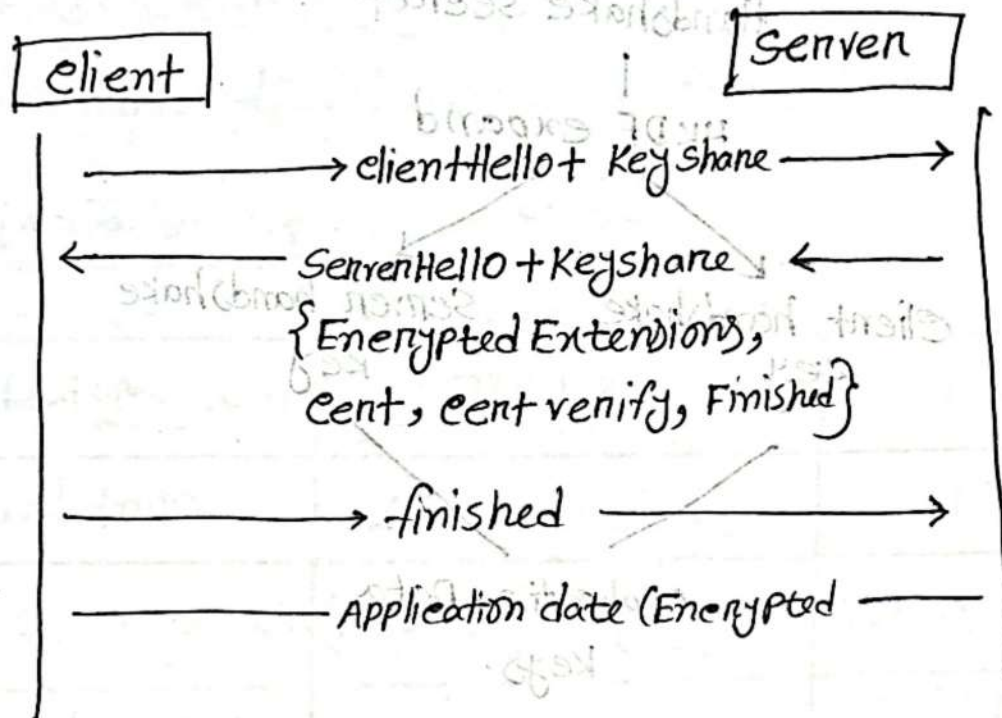
$$c(21, 6) = 69$$

An attacker can't obtain this by simply continuing from the known HMAC.

$$e(30, 6) = 96 \neq 69.$$

So, the noted construction blocks the classic length extension method.

Q38 | TLS 1.3 Handshake diagram



Key exchange

Both sides generate ephemeral key pairs.

client private a $\rightarrow$ client public A

~~client~~ private b $\rightarrow$ server public A
server

using ECDHE / DHE:

$$Z = (B)^a = A^b$$

The shared secret Z itself is never transmitted.

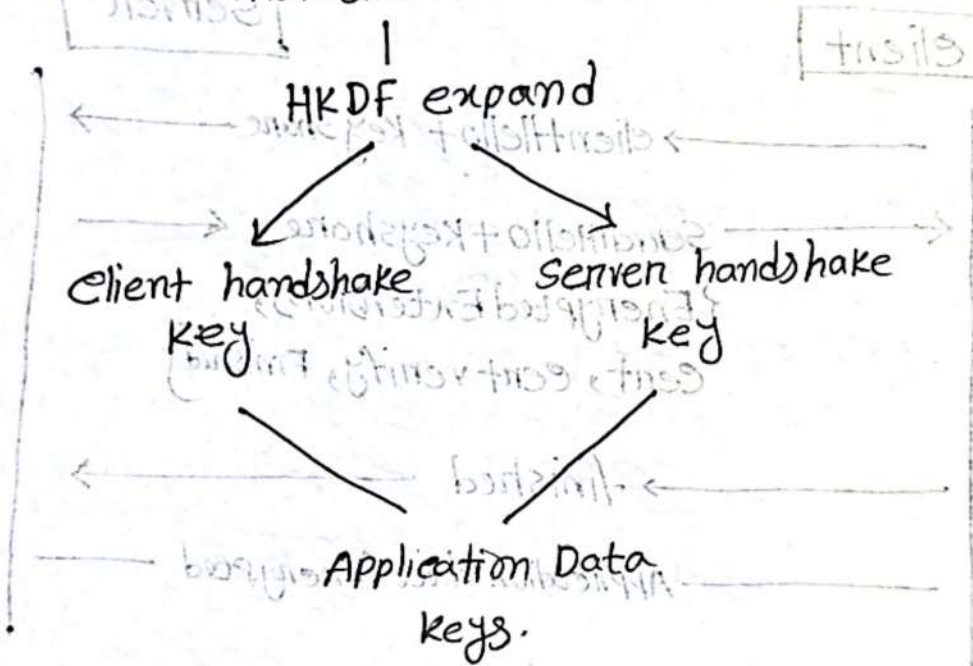
HKDF Key derivation

TLS 1.3 uses HKDF-Extract and HKDF-Expand.

(E)CDHE shared secret

↓
HKDF Extract

Handshake secret



~~The handshake se~~

After the handshake, new secrets are derived for application data encryption.

Finished message:

The finished messages contain a MAC based on the handshake transcript.

it proves that-

- the handshake wasn't modified

both side processes the connect derived secret.

After finished, all application data is encrypted.

TLS 1.3 vs 1.2

Feature	TLS 1.3	TLS 1.2
Round trips	1-RTT	2-RTT
Key exchange	(Ec) DHE only	RSA or (Ec) DHE
Forward secrecy	Always	only if DHE/ EcDHE chosen
Key derivation	HKDF	PRF
Certificate visibility	encrypted	plaintext

So the transmission time of each packet is

important

39] LoRaWAN sensor network, each node sends 1200 bytes packet once every minute.

Data volume per node

Packets per day: $24 \times 60 = 1440$ packets/day

Data per day: $1200 \times 1440 = 1,728,000$ bytes/day.

using $1 \text{ KB} = 1000$ bytes.

$$\frac{1,728,000}{1000} = 1728 \text{ KB/day} \\ = 1.728 \text{ MB/day}$$

So each node generates 1.728 MB/day.

Average Application data rate:

$$\frac{1200 \times 8}{60} = 160 \text{ bits/s}$$

Duty cycle:

A 1% duty cycle means a device can occupy the radio channel for at most:

$$0.01 \times 3600 = 36 \text{ seconds/hour}$$

So, the transmission time of each packet is important.

If one packet takes T seconds / day .

~~on $T < 0.6$ second / day~~

Daily airtime = $1440T$.

For 1% limit:

$1440T \leq 860$ seconds / day

on $T < 0.6$ second / packet.

88401

Aseon vs AES-128:

Aseon is a lightweight authenticated-encryption algorithm selected by NIST for resource-constrained IoT devices.

Feature	Aseon	AES-128
Memory	small-footprint	usually higher in software
Energy	low energy consumption	Higher in software, low with hardware AES.
side channel	Needs constant time/masking protection	Also vulnerable without protection, especially table based AES.
IoT	Designed for constrained devices	general purpose.

Recommendation

For a battery-powered sensor with less than 10 KB RAM, I would choose: Aseon.

Because it's designed for small memory, low power and constrained IoT devices.

Side-channel Resistance:

- power consumption
- Timing
- Electromagnetic leakage.